МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА — Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра вычислительной техники

ОТЧЁТ ПО ПРАКТИЧЕСКИМ РАБОТАМ № 1–3

по дисциплине

«Программирование нейропроцессоров»

Тема: Программная модель, RTL-модель и прототип классификатора падений на ПЛИС

Студенты группы ИКБО-40-23: Рябец М.Ф., Головин А.С., Гришин И.Д.

(Фамилия И. О. студента)

Руководитель работы: Тарасов И.Е.

(Фамилия И. О. руководителя)

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ПОСТАНОВКА ЗАДАЧИ И ДАТАСЕТ SISFALL.....4
2 ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ И ФОРМИРОВАНИЕ ВЫБОРОК.....5
3 АРХИТЕКТУРА СЕТИ И КВАНТОВАНИЕ.....7
4 ОБУЧЕНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ.....8
5 РЕЗУЛЬТАТЫ И ЭТАЛОННЫЙ ОТКЛИК.....9
6 ПОСТАНОВКА ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ № 2.....11
7 РЕАЛИЗАЦИЯ RTL-МОДЕЛИ.....12
8 СИМУЛЯЦИЯ И СВЕРКА С ЭТАЛОНОМ.....13
9 ПРОТОТИП И КОНВЕЙЕРНАЯ АРХИТЕКТУРА.....14
10 ПРОВЕРКА И РЕЗУЛЬТАТЫ РЕАЛИЗАЦИИ.....16
ЗАКЛЮЧЕНИЕ.....19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....20
ПРИЛОЖЕНИЕ А КЛЮЧЕВЫЕ ФРАГМЕНТЫ ПРОГРАММЫ.....21
ПРИЛОЖЕНИЕ Б ИСХОДНЫЙ КОД RTL-МОДЕЛИ.....26
ПРИЛОЖЕНИЕ В КОД ПРОТОТИПА И КОНВЕЙЕРА.....29

ВВЕДЕНИЕ

Падения — одна из основных причин травматизма у пожилых людей, при этом задержка помощи после падения существенно повышает риск осложнений. Носимый инерциальный датчик (акселерометр и гироскоп) позволяет детектировать падение автоматически, но такая система полезна только при работе в реальном времени на энергоэффективном устройстве — то есть на специализированной аппаратной логике, а не на универсальном процессоре с плавающей точкой.

Цель работы — разработать программный эталон классификатора падений, реализовать его на Verilog, проверить совпадение результатов и получить прототип для ПЛИС. После функциональной проверки исследуется влияние регистра между слоями на временные характеристики и использование ресурсов.

Решается задача детектирования падения человека по пяти признакам из данных акселерометра и гироскопа. Сеть $5 \to 8 \to 1$ использует знаковые восьмибитные входы и веса в первом слое; выходной слой работает с бинарными активациями и весами через XNOR и подсчёт совпадений. Такой вычислительный тракт допускает целочисленную реализацию на ПЛИС.

Для достижения цели поставлены следующие задачи:

– получить и разобрать открытый датасет SisFall, реализовать перевод сырых АЦП-кодов датчиков в физические единицы;

– извлечь из сигналов 5 признаков падения и сформировать обучающую и тестовую выборки без утечки данных между испытуемыми;

– спроектировать бинарную нейросеть и обучить её методом BinaryConnect / Straight-Through Estimator на NumPy;

– получить полностью целочисленную (int8, XNOR + popcount) эталонную реализацию сети и оценить потери точности от квантования;

– экспортировать эталонный отклик и веса в формате, готовом к использованию в RTL-тестбенче практической работы № 2.

– реализовать целочисленный классификатор $5 \to 8 \to 1$ на Verilog с загрузкой весов из .mem-файлов;

– выполнить симуляцию RTL-модели и проверить входы, промежуточные состояния и выходы по эталонным векторам.

– проверить верхний модуль с контрольными данными и выполнить размещение и трассировку;

– сравнить исходный и конвейерный вычислительные тракты при одинаковых входах и ограничениях.

1 ПОСТАНОВКА ЗАДАЧИ И ДАТАСЕТ SISFALL

1.1 Требования к системной модели и эталонному отклику

Для детектирования падений разработана программная системная модель нейроускорителя и сформирован эталонный отклик — контрольные пары «вход — выход». По ним проверяется эквивалентность RTL-модели. Вычисления классификатора выполняются в целочисленной арифметике, поэтому результат можно воспроизвести бит в бит в Verilog без операций с плавающей точкой.

1.2 Датасет SisFall

Использован открытый датасет SisFall (A. Sucerquia, J.D. López, J.F. Vargas-Bonilla, SISTEMIC, Universidad de Antioquia, 2016) — один из основных общедоступных наборов данных для задач детектирования падений. Официальный сайт проекта на момент выполнения работы недоступен; архив получен без изменений из зеркала на GitHub (github.com/BIng2325/SisFall).

Датасет содержит записи 38 испытуемых: 23 взрослых (SA01–SA23, 19–30 лет) и 15 пожилых (SE01–SE15, 60–75 лет); 19 видов повседневной активности (коды D01–D19) и 15 видов падений (коды F01–F15), всего 4506 файлов. Данные получены тремя датчиками с частотой опроса 200 Гц: акселерометром ADXL345 (диапазон ±16g, разрешение 13 бит), гироскопом ITG3200 (диапазон ±2000 °/c, разрешение 16 бит) и вторым акселерометром

MMA8451Q (в проекте не используется). Перевод «сырых» АЦП-кодов AD в физические единицы выполняется по формуле, приведённой в Readme датасета:

$$x\,phys = \frac{2 \cdot Range}{2^{Resolution}} \cdot AD \tag{1.1}$$

Пожилые испытуемые (кроме SE06) падения не выполняли по медицинским показаниям — это особенность протокола сбора данных, учтённая при разбиении на обучающую и тестовую выборки (раздел 2.2).

2 ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ И ФОРМИРОВАНИЕ ВЫБОРОК

2.1 Формулы пяти признаков

Для классификации используются пять признаков. Один образец (одно окно анализа) строится вокруг локального пика модуля результирующего ускорения записи — физически обоснованного момента удара при падении или пиковой фазы резкого движения при обычной активности. Для коротких испытаний (12–25 с) берётся одно окно на весь файл; длинные записи ходьбы и бега (100 с) нарезаются на скользящие окна по 4 с с шагом 2 с.

Признак 1 — модуль результирующего ускорения в момент пика:

$$\left|a\left(t_p\right)\right| = \sqrt{a_x^{\,2} + a_y^{\,2} + a_z^{\,2}} \tag{2.1}$$

Признак 3 (пиковая угловая скорость в окрестности пика) вычисляется аналогичной формулой по проекциям гироскопа gx, gy, gz. Признак 2 — угол наклона тела относительно опорной (начальной) ориентации, где a_ref — средний вектор ускорения за первые 0,5 с записи (протокол начинается с неподвижного состояния):

$$tilt = \arccos\left(\frac{a\left(t_p\right) \cdot a_{ref}}{\left|a\left(t_p\right)\right| \cdot \left|a_{ref}\right|}\right) \tag{2.2}$$

Признак 4 — jerk (скорость изменения ускорения), максимум модуля конечной разности в окрестности пика ±0,25 с, f_s = 200 Гц — частота дискретизации:

$$jerk = max\left(f_s \cdot \left|\,|a|(n) - |a|(n-1)\right|\right) \quad (2.3)$$

Признак 5 — дисперсия результирующего ускорения на интервале [t_p+0,5c, t_p+1,5c] после пика: близкая к нулю дисперсия означает, что человек лежит неподвижно (настоящее падение), высокая — что движение продолжается (сел, наклонился). Разделение классов по всем 5 признакам показано на рисунке 2.1: все признаки статистически различимы между классами.

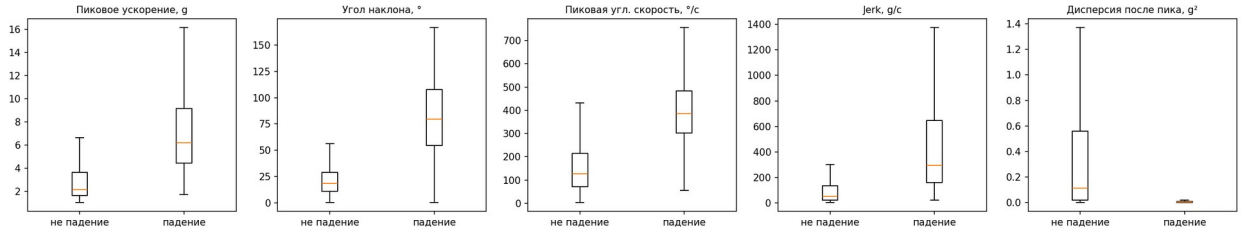


Рисунок 2.1 — Распределение признаков по классам (боксплоты)

Метка окна наследуется от типа исходного файла: любое окно из файла с кодом F* получает метку 1 (падение), из файла с кодом D* — метку 0 (обычная активность) — стандартный для SisFall-подобных исследований способ разметки, так как каждое испытание в протоколе посвящено одной контролируемой активности.

2.2 Разметка и разбиение на train/test

Итоговый набор признаков — 11839 окон (1798 падений, 15,2 %; 10041 не-падений, 84,8 % — ожидаемый дисбаланс классов, так как протокол предусматривает больше сценариев обычной активности, чем падений). Разбиение на обучающую и тестовую выборки выполнено по испытуемым (subject-wise split), а не по отдельным окнам: несколько окон одного человека похожи друг на друга, и попадание одного субъекта одновременно в train и test завысило бы оценку качества. В тест выделены испытуемые SA20, SA21, SA22,

SA23, SE12, SE13, SE14, SE15 (8 из 38, ~21 %), полностью исключённые из обучения (таблица 2.1).

Таблица 2.1 — Разбиение датасета на обучающую и тестовую выборки

Выборка	Испытуемых	Окон	Падений	Не падений
Train	30	9508	1500	8008
Test	8	2331	298	2033

3 АРХИТЕКТУРА СЕТИ И КВАНТОВАНИЕ

3.1 Бинарная нейросеть 5 → 8 → 1

Сеть построена из двух слоёв. Слой A (кодировщик, fixed-point) линейно комбинирует 5 квантованных признаков в 8 промежуточных сигналов и бинаризует их функцией sign — эта операция остаётся в целочисленной (int8×int8 → int32) арифметике, но умножителей здесь всего 40 (5×8), и разрядность мала. Слой B (классификатор, истинно бинарный) линейно комбинирует 8 бинарных сигналов весами ∈{-1,+1} и сравнивается с порогом: скалярное произведение двух ±1-векторов вычисляется без единого умножителя через тождество XNOR + popcount:

$$\mathrm{dot}(a,w) = 2 \cdot popcount(XNOR(a,w)) - H \qquad (3.1)$$

где H = 8 — число нейронов слоя A. Тождество проверено численно на 2000 случайных парах ±1-векторов разной длины; во всех случаях получено полное совпадение. Аппаратная часть ограничена классификатором: слой B реализуется через XNOR, подсчёт единиц и пороговое сравнение, а слой A — через целочисленное умножение и накопление. Всего обучаемых параметров: 40 (W_A, int8) + 8 (b_A, int32) + 8 (W_B, 1 бит) + 1 (b_B, int32) = 57 скалярных величин.

3.2 Fixed-point квантование

Квантование признаков — симметричное, без нулевой точки:

$$q = clip\left(round\left(\frac{x}{scale}\right), -128, 127\right) \qquad (3.2)$$

Масштаб scale для каждого из 5 признаков калибруется по обучающей выборке отдельно (диапазоны физически разные: g, градусы, °/с, g/с, g²) — берётся 99,5-й процентиль модуля значения, а не максимум, чтобы редкие выбросы не «съедали» динамический диапазон 8 бит. Веса слоя A квантуются одним общим масштабом на всю матрицу w_scale_A = max(|W_A|)/127, что корректно именно потому, что входы уже приведены к единому нормированному диапазону на предыдущем шаге. Смещение слоя квантуется в единицах того же аккумулятора (b_A_i32 = round(b_A / (w_scale_A/127))), что сохраняет знак предактивации z — а только знак и используется, так как за слоем A следует sign().

4 ОБУЧЕНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

4.1 BinaryConnect / Straight-Through Estimator

Прямое распространение сигнала через sign() не дифференцируемо, поэтому обучение выполнено методом BinaryConnect / Straight-Through Estimator (Courbariaux et al., 2016) — реализовано вручную на NumPy без сторонних ML-библиотек. Хранятся вещественные «теневые» веса W_A, W_B; на прямом проходе W_B и активации после слоя A бинаризуются знаком; на обратном проходе производная sign(t) заменяется производной hard tanh:

$$\frac{d \cdot sign(t)}{dt} \approx 1,\ |t| \le 1;\ 0,\ \textit{иначе} \qquad (4.1)$$

После каждого шага теневые веса W_B клипуются в [-1, 1] (классический приём BinaryConnect, иначе веса «убегают» и штраф STE перестаёт работать корректно). Ключевые фрагменты прямого и обратного прохода приведены в листингах А.5 и А.6.

4.2 Параметры обучения и сходимость

Параметры обучения: оптимизатор Adam (ручная реализация, lr = 0,05), 300 эпох, размер батча 64, функция потерь — взвешенная бинарная кросс-

энтропия (вес положительного класса ≈ 5,34 = 8008/1500, чтобы скомпенсировать дисбаланс классов). Кривая обучения показана на рисунке 4.1.

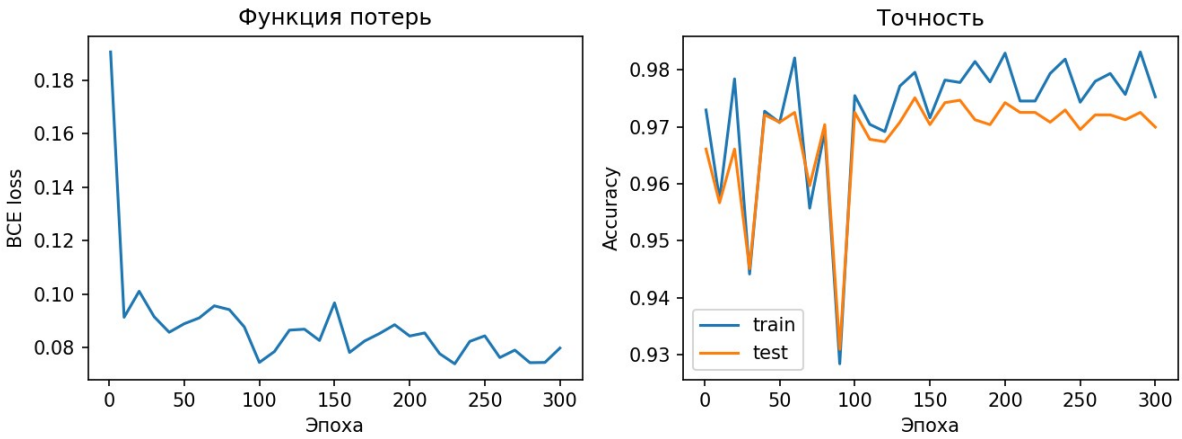


Рисунок 4.1 — Функция потерь и точность на train/test по эпохам

5 РЕЗУЛЬТАТЫ И ЭТАЛОННЫЙ ОТКЛИК

5.1 Метрики качества

Оценка выполнена на тестовой выборке (2331 окно, 8 испытуемых, не встречавшихся при обучении) для двух версий модели: float-модель (вещественные веса, sign() как при обучении) и int8 golden-модель (полностью целочисленная реализация, результат которой и есть эталонный отклик, таблица 5.1).

Таблица 5.1 — Метрики качества на тестовой выборке

Метрика	float-модель	int8 golden-модель
Accuracy	97,0 %	97,0 %
Precision	84,3 %	84,5 %
Recall	94,0 %	93,6 %
F1	0,8889	0,8885
TP / TN / FP / FN	280 / 1981 / 52 / 18	279 / 1982 / 51 / 19

Совпадение решений float- и int8-модели: 99,9 % (расходятся всего 2 из 2331 примеров, в обоих случаях — приграничные значения около порога y≈0), что подтверждает: квантование до int8/бинарных весов практически не ухудшает качество (рисунок 5.1).

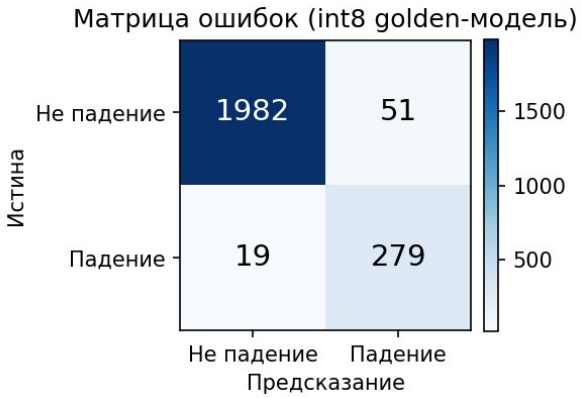


Рисунок 5.1 — Матрица ошибок, int8 эталонная модель

Recall 93,6 % означает, что модель пропускает около 6,4 % реальных падений на тесте. Для системы безопасности снижение числа пропусков возможно, например, путём объединения решений по нескольким последовательным окнам. Precision 84,5 % отражает конкретную особенность выбранных признаков: часть резких, но безопасных движений (быстрая посадка, прыжок) даёт схожий по «остроте» профиль ускорения и jerk, что и настоящее падение — известная проблема чисто акселерометрических детекторов падения, отмеченная и в исходной статье по SisFall.

5.2 Эталонные данные для верификации RTL-модели

Для проверки RTL-модели подготовлен эталонный набор данных:

– полный тестовый набор из 2331 образца: пять квантованных признаков, восьмибитный код слоя A, целочисленная сумма слоя B и итоговый бит;

– контрольный набор из 64 пар «40-битный вход — эталонный выход» для начальной симуляции: 32 падения и 32 обычные активности;

– параметры классификатора: 40 знаковых весов и 8 смещений слоя A, 8 бинарных весов и одно смещение слоя B;

– шестнадцатеричные файлы памяти для загрузки параметров и входных векторов в Verilog через $readmemh.

Контрольный набор проверен повторным декодированием входных слов в int8 и прогоном через целочисленную модель: все 64 ожидаемых выхода воспроизведены.

5.3 Границы аппаратной реализации

«Нейроускоритель», переносимый на ПЛИС, ограничен классификатором (слой A + слой B) — принимает 5 уже готовых квантованных признаков и выдаёт 1 бит класса. Извлечение признаков из сырого потока датчиков (sqrt, arccos, поиск пика, дисперсия) остаётся программным препроцессингом и тестовыми векторами, а не переносится в RTL: эти операции существенно сложнее для аппаратной реализации (нужны CORDIC или LUT-аппроксимации), тогда как сам классификатор специально спроектирован без единого умножителя во втором слое. Такое разделение соответствует практике реальных NPU IP-блоков, где сенсорный DSP-фронтенд отделён от ядра нейроускорителя.

6 ПОСТАНОВКА ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ № 2

6.1 Цель и границы RTL-модели

RTL-модель на Verilog реализует целочисленный классификатор, описанный в разделе 3. Пять рассчитанных и квантованных признаков подаются на вход; поиск пика, вычисление угла, jerk и дисперсии выполняются при программной подготовке данных. Совпадение выходов RTL-модели и программного эталона проверяется симуляцией.

6.2 Интерфейс и формат данных

Входные признаки передаются одним 40-битным словом {x0, x1, x2, x3, x4}; каждый байт является знаковым числом int8 в дополнительном коде. По порядку передаются пиковое ускорение, угол наклона, пиковая угловая скорость, jerk и дисперсия после пика. Результат регистрируется на фронте

тактового сигнала при valid_in=1. Сигнал valid_out показывает, что выходной бит относится к принятому входу. Сброс rst_n синхронный, активный низким уровнем.

Таблица 6.1 — Интерфейс RTL-классификатора

Сигнал	Бит	Назначение
clk	1	Тактовый сигнал
rst_n	1	Синхронный сброс, активный 0
valid_in	1	На входе подготовлено слово признаков
input_features	40	Пять знаковых байтов x0…x4
valid_out	1	Зарегистрированный ответ действителен
fall_detected	1	1 — падение, 0 — обычная активность

Единый проект ПР2–ПР3 создан в Vivado 2022.2 для кристалла Xilinx Artix-7 xc7a100tcsg324-1. В него включены RTL-модули, тестбенчи и файлы памяти с параметрами и контрольными данными.

7 РЕАЛИЗАЦИЯ RTL-МОДЕЛИ

7.1 Слой A: знаковое умножение и накопление

Восемь нейронов слоя A вычисляют $z_h = b_A[h] + \Sigma_j x_j \cdot W_A[j,h]$ для j=0…4. Веса W_A загружаются в порядке j·8+h. Операнды int8 явно трактуются как знаковые; произведение хранится в 16 битах и расширяется до 32 бит перед сложением со смещением int32. Промежуточный бит a_h равен 1 тогда и только тогда, когда $z_h \geq 0$. При нулевой сумме бит также равен 1, как и в программном эталоне.

7.2 Слой B: XNOR и popcount

Восемь бинарных весов W_B сравниваются с битами a_h операцией XNOR. Четырёхбитный счётчик определяет число совпадений от 0 до 8. Итоговая сумма вычисляется как $y = 2 \cdot \text{popcount}(\text{XNOR}(a, W_B)) - 8 + b_B$; выход fall_detected равен 1 при $y \geq 0$. Эта формула бит в бит соответствует программной модели и не требует вещественной арифметики во время вывода.

7.3 Регистры, сброс и загрузка параметров

Комбинационная часть рассчитывает оба слоя для текущего входа. На ближайшем фронте clk при valid_in=1 сохраняются выходной бит и valid_out=1; при отсутствии входа valid_out=0, а последний бит остаётся неизменным. Синхронный сброс обнуляет оба регистра. Веса и смещения загружаются из четырёх .mem-файлов через $readmemh: 40 весов W_A идут в порядке j·8+h, за ними отдельно хранятся 8 смещений b_A, 8 бинарных весов W_B и смещение b_B. Каждая строка файла памяти задаёт одно значение в hex-формате.

8 СИМУЛЯЦИЯ И СВЕРКА С ЭТАЛОНОМ

8.1 Методика верификации

Самопроверяющийся тестбенч подаёт вход до фронта clk и сверяет зарегистрированный выход и valid_out с эталоном. На полном наборе он сравнивает также a_bits и сумму y, что локализует ошибки по слоям. Отдельно проверяются сброс и снятие valid_out. Полный набор содержит 2331 эталонную запись и преобразован в формат памяти для симуляции Verilog.

Ожидаемый выход каждого вектора — решение программной модели, а не истинная метка SisFall. Симуляция проверяет точность аппаратного вычислительного тракта относительно целочисленного эталона.

8.2 Результаты

Таблица 8.1 — Результаты поведенческой симуляции

Набор	Число	Расхождения по выходу	Расхождения внутри слоёв
Vivado XSim, полный набор	2331	0	0 по a_bits и y

Например, для x=[14,16,27,2,0] модель и RTL дали a_bits=10100001, y=−5, fall_detected=0. Расхождений не обнаружено. Это подтверждает функциональную эквивалентность RTL целочисленному эталону на проверенных входах.

В Vivado 2022.2 создан проект для xc7a100tcsg324-1. Поведенческая симуляция XSim завершилась сообщением PASS: 2331 vectors. Синтез успешно завершился; получена логическая схема классификатора (рисунок 8.1). Согласно отчёту ресурсов, использовано 595 Slice LUT, 2 Slice Register, 45 Bonded IOB и 1 BUFGCTRL. Эти показатели относятся к отдельному ядру после синтеза. Размещение и сравнение вариантов выполнены в ПР3.

Таблица 8.2 — Ресурсы отдельного классификатора после синтеза

Ресурс	Количество
Slice LUT	595
Slice Register	2
Bonded IOB	45
BUFGCTRL	1

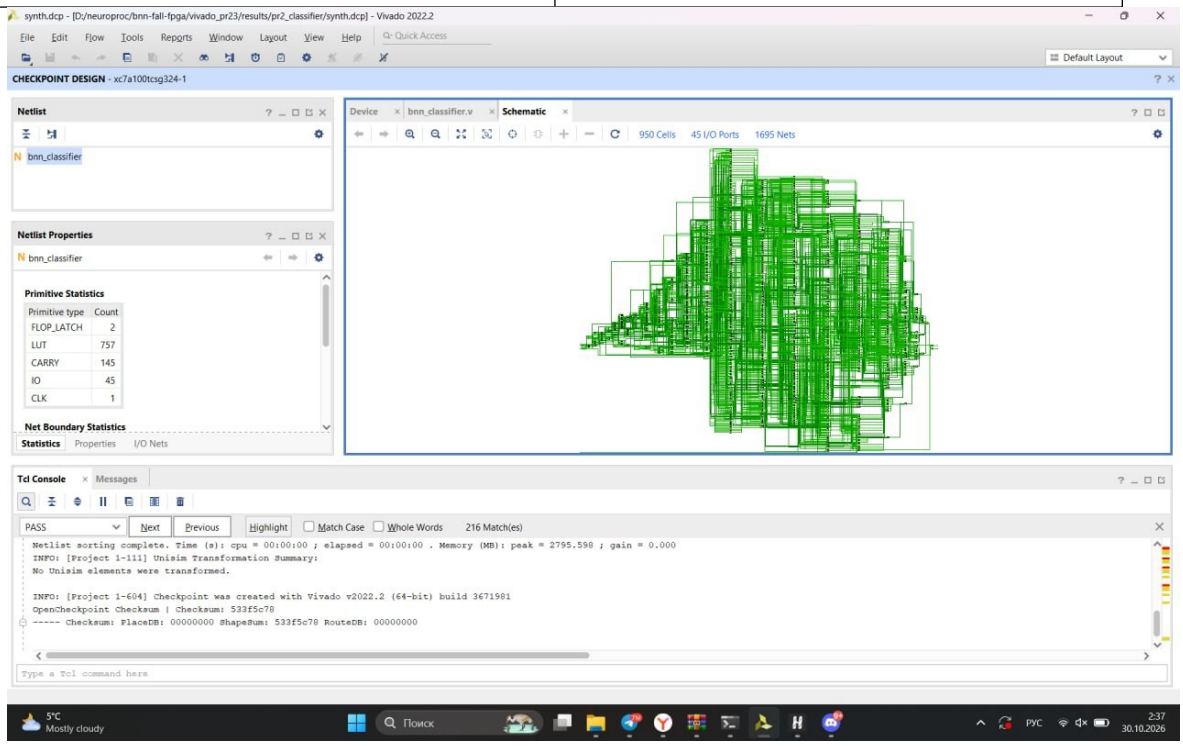


Рисунок 8.1 — Синтезированная схема классификатора

9 ПРОТОТИП И КОНВЕЙЕРНАЯ АРХИТЕКТУРА

9.1 Верхний модуль и контрольные данные

Верхний модуль fpga_top хранит 64 входных слова и 64 ожидаемых бита, подготовленных программной моделью. Автомат IDLE–DRIVE–CHECK подаёт по одному вектору, сверяет ответ классификатора и ведёт счётчики проверок и расхождений. После последней проверки устанавливаются done и pass.

Повторный импульс start запускает проверку заново. Для конвейерного классификатора реализован fpga_top_pipelined: он подаёт новый вектор каждый такт и задерживает индекс эталона вместе с ответом.

Таблица 9.1 — Сигналы верхнего модуля

Сигнал	Разрядность	Назначение
clk, rst_n	1 + 1	Тактирование и синхронный сброс
start	1	Начало контрольного прогона
busy, done, pass	1 + 1 + 1	Ход, завершение и результат
checked_count, error_count	7 + 7	Проверенные векторы и расхождения
last_prediction	1	Последний ответ сети

Контрольный набор подтверждает работу автомата, но он постоянен для синтезатора. Поэтому его схема не используется для оценки выигрыша в частоте: Vivado вправе упростить вычисления под 64 известных слова. Размещение верхнего модуля показано на рисунке 9.1.

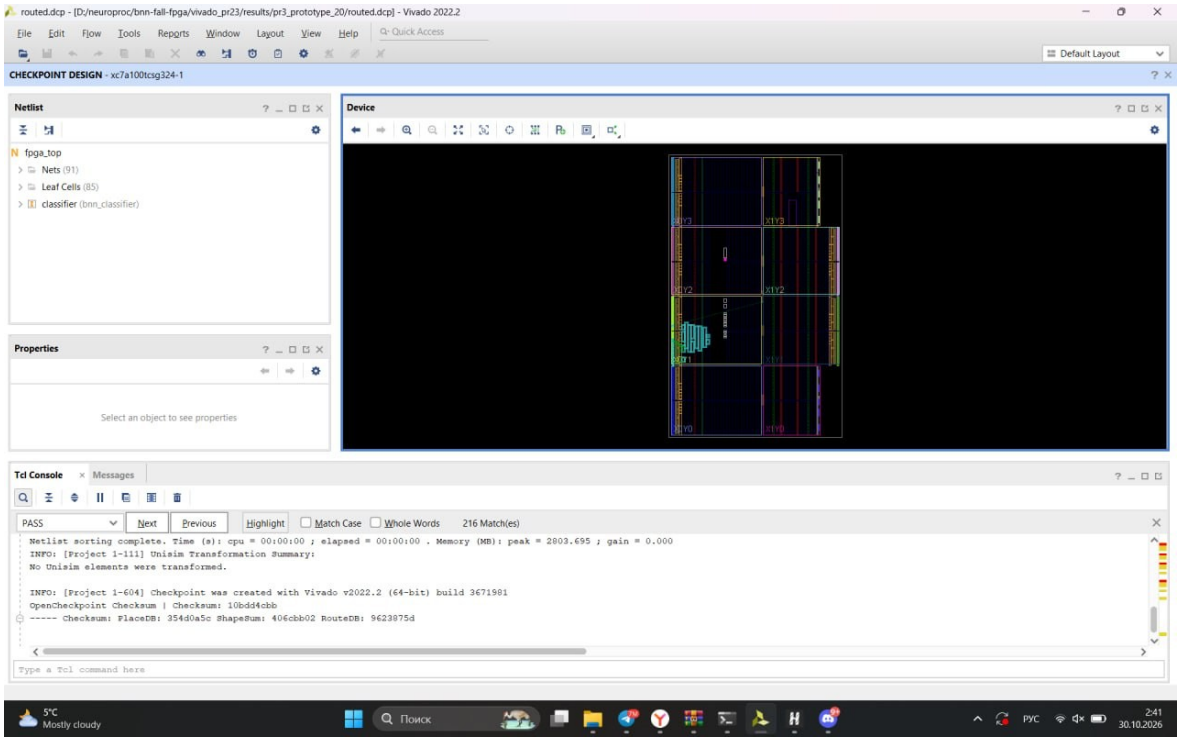


Рисунок 9.1 — Размещение прототипа на кристалле

9.2 Регистры между вычислительными слоями

В исходном классификаторе оба слоя образуют одну комбинационную цепь до регистра ответа. Для измерения этот тракт собран из тех же модулей bnn_layer_a и bnn_layer_b, что и конвейерный вариант. Восемь бит после слоя A

и сигнал valid записываются в регистры; слой B формирует результат для следующего фронта. Регистр поставлен на границе слоёв, а не после каждого арифметического действия. Оба ядра принимают один вектор каждый такт, но конвейерный ответ появляется на один такт позже. Объявление reg в комбинационном always само по себе не означает физический регистр; его создаёт запись на posedge clk.

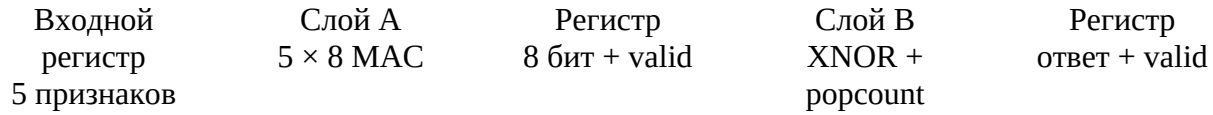


Рисунок 9.2 — Граница регистров между слоями сети

9.3 Условия сравнения

Оба варианта собраны в одном проекте Vivado 2022.2 для xc7a100tcsg324-1. Измерительные верхние модули имеют одинаковый 40-битный вход, входной регистр, интерфейс valid и одни и те же параметры сети. Входы остаются переменными. Для каждого варианта выполнены синтез, размещение и трассировка при периодах 20 и 10 нс. Тактовый период ограничен соответствующим значением. Таким образом, сравнивается именно граница регистров вычислительного тракта. Выводы о таймингах относятся только к ограниченным внутренним синхронным путям.

10 ПРОВЕРКА И РЕЗУЛЬТАТЫ РЕАЛИЗАЦИИ

10.1 Функциональная проверка

Поведенческая симуляция исходного и конвейерного ядер сверила 2331 вход с программным эталоном. Для обоих измерительных верхних модулей те же 2331 ответа проверены также в функциональной симуляции после синтеза. Самопроверяющиеся прототипы прошли контрольный набор из 64 слов; тестбенчи подтвердили обнаружение намеренно изменённого эталонного бита и повторный успешный запуск. При неизменённом эталоне расхождений нет.

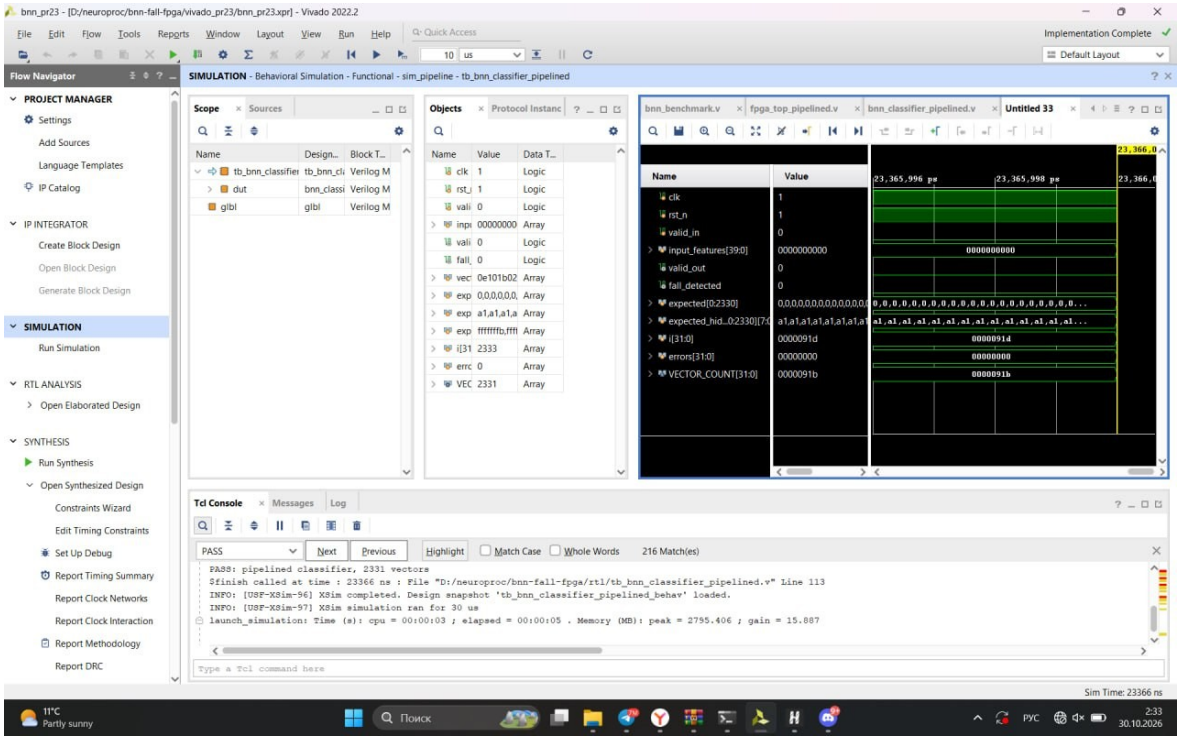


Рисунок 10.1 — Сверка конвейерного классификатора с 2331 эталонным ответом в XSim

10.2 Ресурсы и временные характеристики

Таблица 10.1 — Сравнение вычислительных трактов после трассировки

Вариант	Период, нс	Slice LUT	Регистры	WNS, нс
Исходный	20,000	600	43	6,940
С регистром	20,000	599	52	9,667
Исходный	10,000	613	43	0,055
С регистром	10,000	599	52	1,016

При периоде 20 нс запас по установке вырос с 6,940 до 9,667 нс (на 2,727 нс). При периоде 10 нс оба варианта выдержали ограничение: WNS составил 0,055 и 1,016 нс соответственно. При одинаковом периоде оба ядра принимают один вектор каждый такт. Промежуточный регистр добавил девять триггеров и один такт задержки отдельного ответа. Преимущество конвейера здесь — запас по таймингу, а не увеличение числа ответов за такт. Оба варианта прошли 10 нс (100 МГц) на ограниченных внутренних путях; точная предельная частота по этим двум прогонам не определена.

Таблица 10.2 — Результаты прототипов с контрольной памятью

Верхний модуль	LUT	Регистры	Проложено цепей	WNS при 20 нс
fpga_top	649	29	761/761	5,280
fpga_top_pipelined	653	49	777/777	6,999

Оба прототипа полностью трассированы без ошибок маршрутизации. Эти значения включают автомат и постоянную контрольную память. Они характеризуют верхние модули, но не служат основанием для вывода о скорости вычислительного ядра.

10.3 Границы проверки

Для прототипа не заданы внешние задержки у 2 входов и 18 выходов. Отчёт DRC содержит CFGBVS-1, NSTD-1, UCIO-1. Плата не выбрана, поэтому не задавались физические выводы, стандарты ввода-вывода и напряжение конфигурации. Битовый поток не создавался и испытание на плате не проводилось. Подтверждены симуляция, синтез и физическая реализация для указанного кристалла.

В отдельном монолитном классификаторе Vivado предупредил об удалении одного промежуточного бита при оптимизации. Для сравнения вариантов использованы модульные вычислительные тракты; их ответы проверены на 2331 входе также после синтеза. Все четыре файла параметров сети успешно прочитаны при синтезе.

ЗАКЛЮЧЕНИЕ

Построены программная модель, целочисленный RTL-классификатор и прототип на Artix-7. На 2331 эталонном векторе подтверждено совпадение вычислений с эталоном в поведенческой симуляции; для модульных вариантов — также после синтеза. Верхний модуль проверил 64 контрольных входа, обнаружение ошибки и повторный запуск. Размещение и трассировка в Vivado 2022.2 завершены для исходного и конвейерного вариантов. При периоде 20 нс запас по установке вырос с 6,940 до 9,667 нс (на 2,727 нс). При периоде 10 нс оба варианта выдержали ограничение: WNS составил 0,055 и 1,016 нс соответственно. Оценка относится к внутренним синхронным путям; назначение выводов и проверка на плате не выполнялись.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Практическая работа № 1 «Разработка системной модели нейроускорителя»: задание по дисциплине «Программирование нейропроцессоров». — М.: РТУ МИРЭА, 2026. — 2 с.

2. Sucerquia A., López J.D., Vargas-Bonilla J.F. SisFall: A Fall and Movement Dataset // Sensors. — 2017. — Vol. 17, № 1. — Art. 198.

3. Courbariaux M., Bengio Y., David J.-P. BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations // Advances in Neural Information Processing Systems 28 (NeurIPS 2015). — 2015. — P. 3123–3131.

4. Rastegari M., Ordonez V., Redmon J., Farhadi A. XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks // European Conference on Computer Vision (ECCV 2016). — 2016. — P. 525–542.

5. Правила оформления курсовой работы (проекта), ВКР и письменных работ бакалавра и магистра. — М.: РТУ МИРЭА, Институт информационных технологий, 2023. — 24 с.

6. NumPy Developers. NumPy Reference. Version 2.4. — 2026. — URL: https://numpy.org/doc/stable/reference/ (дата обращения: 28.09.2026).

7. SisFall dataset mirror. — URL: https://github.com/BIng2325/SisFall (дата обращения: 28.09.2026).

8. AMD. Vivado Design Suite User Guide: Logic Simulation (UG900), version 2022.2. — URL: https://docs.amd.com/r/2022.2-English/ug900-vivado-logic-simulation/ (дата обращения: 03.10.2026).

9. AMD. Vivado Design Suite User Guide: Synthesis (UG901), version 2022.2. — URL: https://docs.amd.com/r/2022.2-English/ug901-vivado-synthesis/ (дата обращения: 03.10.2026).

10. AMD. Vivado Design Suite User Guide: Implementation (UG904), version 2022.2. — URL: https://docs.amd.com/r/2022.2-English/ug904-vivado-implementation/ (дата обращения: 28.10.2026).

Приложение А

Ключевые фрагменты программы

Ключевые алгоритмы программной модели представлены следующими фрагментами.

Листинг А.1 — Перевод АЦП-кодов SisFall в физические единицы

```
def load_recording(path: Path) -> Recording:
    code, subject, trial = parse_filename(path)
    raw_text = path.read_text().replace(";", "")
    raw = np.genfromtxt(raw_text.splitlines(), delimiter=",",
dtype=np.float64)
    if raw.ndim == 1:
        raw = raw.reshape(1, -1)
    if raw.shape[1] != 9:
        raise ValueError(f"{path}: expected 9 columns, got {raw.shape[1]}")

    acc_g = raw[:, 0:3] * ACC_SCALE
    gyro_dps = raw[:, 3:6] * GYRO_SCALE

    return Recording(
        path=path,
        code=code,
        subject=subject,
        trial=trial,
        is_fall=code in FALL_CODES,
        acc_g=acc_g,
        gyro_dps=gyro_dps,
    )
```

Листинг А.2 — Извлечение 5 признаков из одного окна анализа

```
def _extract_one_window(rec: Recording, res_acc: np.ndarray, res_gyro:
np.ndarray,
                        ref_vec: np.ndarray, search_lo: int, search_hi: int)
-> FeatureSample | None:
    n = rec.n_samples
    search_hi = min(search_hi, n)
    if search_hi - search_lo < 10:
        return None

    local_peak = int(np.argmax(res_acc[search_lo:search_hi]))
    peak_idx = search_lo + local_peak

    peak_acc_g = float(res_acc[peak_idx])

    a_peak = rec.acc_g[peak_idx]
    cos_theta = np.dot(a_peak, ref_vec) / (np.linalg.norm(a_peak) *
np.linalg.norm(ref_vec) + 1e-9)
    tilt_deg = float(np.degrees(np.arccos(np.clip(cos_theta, -1.0, 1.0))))

    gyro_nb = int(GYRO_NEIGHBORHOOD_S * FS)
    g_lo, g_hi = max(0, peak_idx - gyro_nb), min(n, peak_idx + gyro_nb)
    peak_gyro_dps = float(np.max(res_gyro[g_lo:g_hi]))

    jerk_nb = int(JERK_NEIGHBORHOOD_S * FS)
    j_lo, j_hi = max(1, peak_idx - jerk_nb), min(n, peak_idx + jerk_nb)
    if j_hi - j_lo < 2:
        jerk_g_per_s = 0.0
    else:
        jerk_series = np.abs(np.diff(res_acc[j_lo - 1:j_hi])) * FS
        jerk_g_per_s = float(np.max(jerk_series))

    p_lo = peak_idx + int(POST_PEAK_START_S * FS)
    p_hi = peak_idx + int(POST_PEAK_END_S * FS)
    p_hi = min(p_hi, n)
    if p_hi - p_lo < MIN_POST_PEAK_SAMPLES:
        return None
    post_peak_var_g2 = float(np.var(res_acc[p_lo:p_hi]))

    features = np.array([peak_acc_g, tilt_deg, peak_gyro_dps, jerk_g_per_s,
post_peak_var_g2])
    return FeatureSample(
        features=features,
        label=int(rec.is_fall),
        subject=rec.subject,
        code=rec.code,
        trial=rec.trial,
        peak_idx=peak_idx,
    )
```

Листинг А.3 — Обход записей и построение выборок с меткой

```
def extract_samples(rec: Recording) -> list[FeatureSample]:
    n = rec.n_samples
    res_acc = _resultant(rec.acc_g)
    res_gyro = _resultant(rec.gyro_dps)
    calib_n = max(1, min(n, int(CALIBRATION_S * FS)))
    ref_vec = np.mean(rec.acc_g[:calib_n], axis=0)
    if np.linalg.norm(ref_vec) < 1e-6:
        ref_vec = np.array([0.0, 0.0, 1.0])

    samples: list[FeatureSample] = []
    if rec.code in LONG_TRIAL_CODES:
        start = 0
        while start < n:
            hi = min(start + WINDOW_N, n)
            s = _extract_one_window(rec, res_acc, res_gyro, ref_vec, start,
hi)
            if s is not None:
                samples.append(s)
            if hi >= n:
                break
            start += STRIDE_N
    else:
        s = _extract_one_window(rec, res_acc, res_gyro, ref_vec, 0, n)
        if s is not None:
            samples.append(s)
    return samples
```

Листинг А.4 — Калибровка масштаба int8-квантования признаков

```
def calibrate(x: np.ndarray, percentile: float = CALIBRATION_PERCENTILE) ->
QuantParams:
    """x: (N, C). Возвращает по одному масштабу на каждый из C столбцов."""
    x = np.atleast_2d(x)
    abs_max = np.percentile(np.abs(x), percentile, axis=0)
    abs_max = np.maximum(abs_max, 1e-8)
    scale = abs_max / INT8_MAX
    return QuantParams(scale=scale)
```

Листинг А.5 — Прямой проход BNN (бинаризация активаций и весов слоя B)

```
    def forward(self, X: np.ndarray):
        z = X @ self.W_A + self.b_A          # (N, H)
        a = sign(z)                             # (N, H) in {-1,+1}
        Wb = sign(self.W_B)                      # (H,)   in {-1,+1}
        y = a @ Wb + self.b_B                     # (N,)
        p = sigmoid(y)
        cache = dict(X=X, z=z, a=a, Wb=Wb, y=y)
        return p, cache
```

Листинг А.6 — Обратный проход BNN (STE-градиенты)

```
    def backward(self, cache: dict, label: np.ndarray, sample_weight:
np.ndarray | None = None):
        X, z, a, Wb = cache["X"], cache["z"], cache["a"], cache["Wb"]
        N = X.shape[0]
        p = sigmoid(cache["y"])
        w = np.ones(N) if sample_weight is None else sample_weight
        w = w / w.sum()  # нормировка, чтобы масштаб градиента не зависел от
размера батча

        dLdy = (p - label) * w                    # (N,)  d(BCE)/dy при y =
логит
        dLdb_B = np.sum(dLdy)
        dLdWb = a.T @ dLdy                         # (H,)
        dLdW_B = dLdWb * (np.abs(self.W_B) <= 1.0)   # STE-клип

        dLda = np.outer(dLdy, Wb)                    # (N, H)
        dLdz = dLda * (np.abs(z) <= 1.0)               # STE-клип по
предактивации слоя A

        dLdW_A = X.T @ dLdz                             # (n_in, H)
        dLdb_A = np.sum(dLdz, axis=0)

        return dict(W_A=dLdW_A, b_A=dLdb_A, W_B=dLdW_B, b_B=dLdb_B)
```

Листинг А.7 — Квантование обученной модели в int8 / бинарные веса

```
def quantize_model(W_A: np.ndarray, b_A: np.ndarray, W_B: np.ndarray, b_B:
float,
                   in_scale: np.ndarray) -> GoldenWeights:
    w_scale_A = max(float(np.max(np.abs(W_A))) / INT8_MAX, 1e-8)
    W_A_i8 = np.clip(np.round(W_A / w_scale_A), INT8_MIN,
INT8_MAX).astype(np.int32)
    b_A_i32 = np.round(b_A / (w_scale_A / INT8_MAX)).astype(np.int64)

    W_B_sign = np.where(W_B >= 0, 1, -1)
    W_B_bit = (W_B_sign > 0).astype(np.int8)
    b_B_i32 = int(np.round(b_B))

    return GoldenWeights(
        in_scale=in_scale.astype(np.float64),
        W_A_i8=W_A_i8.astype(np.int32),
        b_A_i32=b_A_i32.astype(np.int64),
        w_scale_A=w_scale_A,
        W_B_bit=W_B_bit,
        b_B_i32=b_B_i32,
    )
```

Листинг А.8 — Целочисленный прямой проход эталонной модели

```
def forward_int(gw: GoldenWeights, X_i8: np.ndarray):
    """X_i8: (N, n_in) int. Возвращает (z_i32, a_bit, y_i32, label)."""
    X_i8 = np.atleast_2d(X_i8).astype(np.int64)
    z_i32 = X_i8 @ gw.W_A_i8.astype(np.int64) + gw.b_A_i32   # (N, H)
    a_bit = (z_i32 >= 0).astype(np.int8)                       # (N, H)

    y_i32 = xnor_popcount_dot(a_bit, np.broadcast_to(gw.W_B_bit, a_bit.shape))
+ gw.b_B_i32
    label = (y_i32 >= 0).astype(np.int32)
    return z_i32, a_bit, y_i32, label
```

Листинг А.9 — Тождество XNOR + popcount для ±1-векторов

```
def xnor_popcount_dot(a_bit: np.ndarray, w_bit: np.ndarray) -> np.ndarray:
    """dot(a_pm1, w_pm1) через XNOR + popcount, a_bit/w_bit in {0,1}."""
    xnor = 1 - (a_bit ^ w_bit)              # 1, если биты совпали (оба +1 или
оба -1)
    H = w_bit.shape[-1]
    popcount = np.sum(xnor, axis=-1)
    return 2 * popcount - H
```

Приложение Б

Исходный код RTL-модели

Приведён полный код классификатора и самопроверяющегося тестбенча, использованных в симуляции. Комментарии опущены.

Листинг Б.1 — RTL-модуль классификатора

```
`timescale 1ns / 1ps

module bnn_classifier #(
    parameter W_A_FILE = "weights_layerA_W.mem",
    parameter B_A_FILE = "weights_layerA_b.mem",
    parameter W_B_FILE = "weights_layerB_W.mem",
    parameter B_B_FILE = "weights_layerB_b.mem"
) (
    input  wire        clk,
    input  wire        rst_n,
    input  wire        valid_in,
    input  wire [39:0] input_features,
    output reg         valid_out,
    output reg         fall_detected
);
    reg [7:0]  w_a [0:39];
    reg [31:0] b_a [0:7];
    reg [0:0]  w_b [0:7];
    reg [31:0] b_b [0:0];

    initial begin
        $readmemh(W_A_FILE, w_a);
        $readmemh(B_A_FILE, b_a);
        $readmemh(W_B_FILE, w_b);
        $readmemh(B_B_FILE, b_b);
    end

    reg signed [15:0] product;
    reg signed [31:0] accumulator;
    reg [7:0] hidden_bits;
    reg [3:0] matching_bits;
    reg signed [31:0] score;
    reg prediction_next;
    integer h;
    integer j;

    always @* begin
        product = 16'sd0;
        accumulator = 32'sd0;
        hidden_bits = 8'b0;
        matching_bits = 4'b0;
        score = 32'sd0;
        prediction_next = 1'b0;

        for (h = 0; h < 8; h = h + 1) begin
            accumulator = $signed(b_a[h]);
            for (j = 0; j < 5; j = j + 1) begin
                product = $signed(input_features[39 - 8*j -: 8])
                        * $signed(w_a[j*8 + h]);
                accumulator = accumulator + {{16{product[15]}}, product};
            end
            hidden_bits[h] = (accumulator >= 32'sd0);
        end

        for (h = 0; h < 8; h = h + 1)
            matching_bits = matching_bits + (hidden_bits[h] ~^ w_b[h][0]);

        score = ($signed({28'b0, matching_bits}) <<< 1)
              - 32'sd8 + $signed(b_b[0]);
        prediction_next = (score >= 32'sd0);
```

```
    end

    always @(posedge clk) begin
        if (!rst_n) begin
            valid_out <= 1'b0;
            fall_detected <= 1'b0;
        end else begin
            valid_out <= valid_in;
            if (valid_in)
                fall_detected <= prediction_next;
        end
    end
endmodule
```

Листинг Б.2 — Самопроверяющийся тестбенч

```
`timescale 1ns / 1ps

module tb_bnn_classifier;
`ifdef FULL_TEST
    localparam VECTOR_COUNT = 2331;
`elsif EDGE_TEST
    localparam VECTOR_COUNT = 14;
`else
    localparam VECTOR_COUNT = 64;
`endif

    reg clk = 1'b0;
    always #5 clk = ~clk;

    reg rst_n = 1'b0;
    reg valid_in = 1'b0;
    reg [39:0] input_features = 40'b0;
    wire valid_out;
    wire fall_detected;

    reg [39:0] vectors [0:VECTOR_COUNT-1];
    reg expected [0:VECTOR_COUNT-1];
`ifdef FULL_TEST
    reg [7:0] expected_hidden [0:VECTOR_COUNT-1];
    reg [31:0] expected_score [0:VECTOR_COUNT-1];
`elsif EDGE_TEST
    reg [7:0] expected_hidden [0:VECTOR_COUNT-1];
    reg [31:0] expected_score [0:VECTOR_COUNT-1];
`endif

    integer i;
    integer errors;

    bnn_classifier dut (
        .clk(clk),
        .rst_n(rst_n),
        .valid_in(valid_in),
        .input_features(input_features),
        .valid_out(valid_out),
        .fall_detected(fall_detected)
    );

    initial begin
`ifdef FULL_TEST
        $readmemh("tb_inputs_full.mem", vectors);
        $readmemh("tb_expected_full.mem", expected);
        $readmemh("tb_hidden_full.mem", expected_hidden);
        $readmemh("tb_score_full.mem", expected_score);
`elsif EDGE_TEST
        $readmemh("tb_inputs_edge.mem", vectors);
        $readmemh("tb_expected_edge.mem", expected);
        $readmemh("tb_hidden_edge.mem", expected_hidden);
        $readmemh("tb_score_edge.mem", expected_score);
`else
        $readmemh("tb_inputs_64.mem", vectors);
        $readmemh("tb_expected_64.mem", expected);
```

```
`endif

        errors = 0;
        repeat (2) @(negedge clk);
        if (valid_out !== 1'b0 || fall_detected !== 1'b0)
            $fatal(1, "synchronous reset did not clear outputs");
        rst_n = 1'b1;

        for (i = 0; i < VECTOR_COUNT; i = i + 1) begin
            @(negedge clk);
            input_features = vectors[i];
            valid_in = 1'b1;
            #1;
`ifdef FULL_TEST
            if (dut.hidden_bits !== expected_hidden[i]) begin
                $display("FAIL vector %0d: hidden=%02h expected=%02h",
                         i, dut.hidden_bits, expected_hidden[i]);
                errors = errors + 1;
            end
            if (dut.score !== $signed(expected_score[i])) begin
                $display("FAIL vector %0d: score=%0d expected=%0d",
                         i, dut.score, $signed(expected_score[i]));
                errors = errors + 1;
            end
`elsif EDGE_TEST
            if (dut.hidden_bits !== expected_hidden[i]) begin
                $display("FAIL edge %0d: hidden=%02h expected=%02h",
                         i, dut.hidden_bits, expected_hidden[i]);
                errors = errors + 1;
            end
            if (dut.score !== $signed(expected_score[i])) begin
                $display("FAIL edge %0d: score=%0d expected=%0d",
                         i, dut.score, $signed(expected_score[i]));
                errors = errors + 1;
            end
`endif
            @(posedge clk);
            #1;
            if (valid_out !== 1'b1 || fall_detected !== expected[i]) begin
                $display("FAIL vector %0d: valid=%b output=%b expected=%b",
                         i, valid_out, fall_detected, expected[i]);
                errors = errors + 1;
            end
        end

        @(negedge clk);
        valid_in = 1'b0;
        @(posedge clk);
        #1;
        if (valid_out !== 1'b0) begin
            $display("FAIL: valid_out did not clear after valid_in");
            errors = errors + 1;
        end

        if (errors != 0)
            $fatal(1, "%0d failures in %0d vectors", errors, VECTOR_COUNT);
        $display("PASS: %0d vectors; output, valid signal and golden model agree",
VECTOR_COUNT);
        $finish;
    end
endmodule
```

Приложение В

Код прототипа и конвейерного варианта

Приведены полные тексты вычислительных и верхних модулей, использованных для реализации и сравнения. Комментарии опущены.

Листинг В.1 — Слой А

```
`timescale 1ns / 1ps

module bnn_layer_a #(
    parameter W_FILE = "weights_layerA_W.mem",
    parameter B_FILE = "weights_layerA_b.mem"
) (
    input  wire [39:0] input_features,
    output reg  [7:0]  hidden_bits
);
    reg [7:0]  weights [0:39];
    reg [31:0] biases [0:7];

    initial begin
        $readmemh(W_FILE, weights);
        $readmemh(B_FILE, biases);
    end

    reg signed [15:0] product;
    reg signed [31:0] accumulator;
    integer h;
    integer j;

    always @* begin
        product = 16'sd0;
        accumulator = 32'sd0;
        hidden_bits = 8'b0;
        for (h = 0; h < 8; h = h + 1) begin
            accumulator = $signed(biases[h]);
            for (j = 0; j < 5; j = j + 1) begin
                product = $signed(input_features[39 - 8*j -: 8])
                        * $signed(weights[j*8 + h]);
                accumulator = accumulator + {{16{product[15]}}, product};
            end
            hidden_bits[h] = (accumulator >= 32'sd0);
        end
    end
endmodule
```

Листинг В.2 — Слой В

```
`timescale 1ns / 1ps

module bnn_layer_b #(
    parameter W_FILE = "weights_layerB_W.mem",
    parameter B_FILE = "weights_layerB_b.mem"
) (
    input  wire [7:0]         hidden_bits,
    output reg  signed [31:0] score,
    output reg                prediction
);
    reg [0:0]  weights [0:7];
    reg [31:0] biases [0:0];

    initial begin
        $readmemh(W_FILE, weights);
        $readmemh(B_FILE, biases);
    end

    reg [3:0] matching_bits;
    integer h;
```

```
    always @* begin
        matching_bits = 4'b0;
        for (h = 0; h < 8; h = h + 1)
            matching_bits = matching_bits + (hidden_bits[h] ~^ weights[h][0]);
        score = ($signed({28'b0, matching_bits}) <<< 1)
              - 32'sd8 + $signed(biases[0]);
        prediction = (score >= 32'sd0);
    end
endmodule
```

Листинг В.3 — Классификатор без промежуточного регистра

```
`timescale 1ns / 1ps

module bnn_classifier_modular #(
    parameter W_A_FILE = "weights_layerA_W.mem",
    parameter B_A_FILE = "weights_layerA_b.mem",
    parameter W_B_FILE = "weights_layerB_W.mem",
    parameter B_B_FILE = "weights_layerB_b.mem"
) (
    input  wire        clk,
    input  wire        rst_n,
    input  wire        valid_in,
    input  wire [39:0] input_features,
    output reg         valid_out,
    output reg         fall_detected
);
    wire [7:0] hidden_bits;
    wire signed [31:0] score;
    wire prediction_next;

    bnn_layer_a #(.W_FILE(W_A_FILE), .B_FILE(B_A_FILE)) layer_a (
        .input_features(input_features),
        .hidden_bits(hidden_bits)
    );

    bnn_layer_b #(.W_FILE(W_B_FILE), .B_FILE(B_B_FILE)) layer_b (
        .hidden_bits(hidden_bits),
        .score(score),
        .prediction(prediction_next)
    );

    always @(posedge clk) begin
        if (!rst_n) begin
            valid_out <= 1'b0;
            fall_detected <= 1'b0;
        end else begin
            valid_out <= valid_in;
            if (valid_in)
                fall_detected <= prediction_next;
        end
    end
endmodule
```

Листинг В.4 — Конвейерный классификатор

```
`timescale 1ns / 1ps

module bnn_classifier_pipelined #(
    parameter W_A_FILE = "weights_layerA_W.mem",
    parameter B_A_FILE = "weights_layerA_b.mem",
    parameter W_B_FILE = "weights_layerB_W.mem",
    parameter B_B_FILE = "weights_layerB_b.mem"
) (
    input  wire        clk,
    input  wire        rst_n,
    input  wire        valid_in,
    input  wire [39:0] input_features,
    output reg         valid_out,
    output reg         fall_detected
);
    wire [7:0] hidden_next;
    reg [7:0] hidden_q;
```

```
    reg valid_hidden_q;
    wire signed [31:0] score;
    wire prediction_next;

    bnn_layer_a #(.W_FILE(W_A_FILE), .B_FILE(B_A_FILE)) layer_a (
        .input_features(input_features),
        .hidden_bits(hidden_next)
    );

    bnn_layer_b #(.W_FILE(W_B_FILE), .B_FILE(B_B_FILE)) layer_b (
        .hidden_bits(hidden_q),
        .score(score),
        .prediction(prediction_next)
    );

    always @(posedge clk) begin
        if (!rst_n) begin
            hidden_q <= 8'b0;
            valid_hidden_q <= 1'b0;
            valid_out <= 1'b0;
            fall_detected <= 1'b0;
        end else begin
            hidden_q <= hidden_next;
            valid_hidden_q <= valid_in;
            valid_out <= valid_hidden_q;
            if (valid_hidden_q)
                fall_detected <= prediction_next;
        end
    end
endmodule
```

Листинг B.5 — Прототип с контрольными данными

```
`timescale 1ns / 1ps

module fpga_top (
    input  wire       clk,
    input  wire       rst_n,
    input  wire       start,
    output reg        busy,
    output reg        done,
    output reg        pass,
    output reg [6:0]  checked_count,
    output reg [6:0]  error_count,
    output reg        last_prediction
);
    localparam [1:0] IDLE = 2'd0;
    localparam [1:0] DRIVE = 2'd1;
    localparam [1:0] CHECK = 2'd2;

    reg [1:0] state;
    reg [5:0] vector_index;
    reg [39:0] vectors [0:63];
    reg [0:0] expected [0:63];

    initial begin
        $readmemh("tb_inputs_64.mem", vectors);
        $readmemh("tb_expected_64.mem", expected);
    end

    wire [39:0] input_features = vectors[vector_index];
    wire valid_in = (state == DRIVE);
    wire valid_out;
    wire fall_detected;
    wire mismatch = (valid_out != 1'b1) ||
                    (fall_detected != expected[vector_index][0]);

    bnn_classifier classifier (
        .clk(clk),
        .rst_n(rst_n),
        .valid_in(valid_in),
        .input_features(input_features),
```

```
        .valid_out(valid_out),
        .fall_detected(fall_detected)
    );

    always @(posedge clk) begin
        if (!rst_n) begin
            state <= IDLE;
            vector_index <= 6'd0;
            busy <= 1'b0;
            done <= 1'b0;
            pass <= 1'b0;
            checked_count <= 7'd0;
            error_count <= 7'd0;
            last_prediction <= 1'b0;
        end else begin
            case (state)
                IDLE: begin
                    if (start) begin
                        vector_index <= 6'd0;
                        busy <= 1'b1;
                        done <= 1'b0;
                        pass <= 1'b0;
                        checked_count <= 7'd0;
                        error_count <= 7'd0;
                        state <= DRIVE;
                    end
                end
                DRIVE: state <= CHECK;
                CHECK: begin
                    checked_count <= checked_count + 7'd1;
                    last_prediction <= fall_detected;
                    if (mismatch)
                        error_count <= error_count + 7'd1;
                    if (vector_index == 6'd63) begin
                        busy <= 1'b0;
                        done <= 1'b1;
                        pass <= (error_count == 7'd0) && !mismatch;
                        state <= IDLE;
                    end else begin
                        vector_index <= vector_index + 6'd1;
                        state <= DRIVE;
                    end
                end
                default: state <= IDLE;
            endcase
        end
    end
endmodule
```

Листинг B.6 — Конвейерный прототип

```
`timescale 1ns / 1ps

module fpga_top_pipelined (
    input  wire       clk,
    input  wire       rst_n,
    input  wire       start,
    output reg        busy,
    output reg        done,
    output reg        pass,
    output reg [6:0]  checked_count,
    output reg [6:0]  error_count,
    output reg        last_prediction
);
    localparam [1:0] IDLE = 2'd0;
    localparam [1:0] SEND = 2'd1;
    localparam [1:0] DRAIN = 2'd2;

    reg [1:0] state;
    reg [5:0] send_index;
    reg [5:0] check_index_q0;
    reg [5:0] check_index_q1;
```

```
reg [39:0] vectors [0:63];
reg [0:0] expected [0:63];

initial begin
    $readmemh("tb_inputs_64.mem", vectors);
    $readmemh("tb_expected_64.mem", expected);
end

wire [39:0] input_features = vectors[send_index];
wire valid_in = (state == SEND);
wire valid_out;
wire fall_detected;
wire mismatch = (fall_detected != expected[check_index_q1][0]);

bnn_classifier_pipelined classifier (
    .clk(clk),
    .rst_n(rst_n),
    .valid_in(valid_in),
    .input_features(input_features),
    .valid_out(valid_out),
    .fall_detected(fall_detected)
);

always @(posedge clk) begin
    if (!rst_n) begin
        state <= IDLE;
        send_index <= 6'd0;
        check_index_q0 <= 6'd0;
        check_index_q1 <= 6'd0;
        busy <= 1'b0;
        done <= 1'b0;
        pass <= 1'b0;
        checked_count <= 7'd0;
        error_count <= 7'd0;
        last_prediction <= 1'b0;
    end else begin
        check_index_q0 <= send_index;
        check_index_q1 <= check_index_q0;
        case (state)
            IDLE: begin
                if (start) begin
                    send_index <= 6'd0;
                    busy <= 1'b1;
                    done <= 1'b0;
                    pass <= 1'b0;
                    checked_count <= 7'd0;
                    error_count <= 7'd0;
                    state <= SEND;
                end
            end
            SEND: begin
                if (send_index == 6'd63)
                    state <= DRAIN;
                else
                    send_index <= send_index + 6'd1;
            end
            DRAIN: begin end
            default: state <= IDLE;
        endcase
        if (valid_out && state != IDLE) begin
            checked_count <= checked_count + 7'd1;
            last_prediction <= fall_detected;
            if (mismatch)
                error_count <= error_count + 7'd1;
            if (checked_count == 7'd63) begin
                busy <= 1'b0;
                done <= 1'b1;
                pass <= (error_count == 7'd0) && !mismatch;
                state <= IDLE;
            end
        end
```

```
        end
    end
endmodule
```

Листинг В.7 — Верхние модули для сравнения

```
`timescale 1ns / 1ps

module bnn_benchmark_shell #(
    parameter PIPELINED = 0
) (
    input  wire        clk,
    input  wire        rst_n,
    input  wire        valid_in,
    input  wire [39:0] input_features,
    output wire        valid_out,
    output wire        fall_detected
);
    reg [39:0] input_q;
    reg valid_q;

    always @(posedge clk) begin
        if (!rst_n) begin
            input_q <= 40'b0;
            valid_q <= 1'b0;
        end else begin
            input_q <= input_features;
            valid_q <= valid_in;
        end
    end

    generate
        if (PIPELINED) begin : with_register
            bnn_classifier_pipelined classifier (
                .clk(clk), .rst_n(rst_n), .valid_in(valid_q),
                .input_features(input_q),
                .valid_out(valid_out), .fall_detected(fall_detected)
            );
        end else begin : without_register
            bnn_classifier_modular classifier (
                .clk(clk), .rst_n(rst_n), .valid_in(valid_q),
                .input_features(input_q),
                .valid_out(valid_out), .fall_detected(fall_detected)
            );
        end
    endgenerate
endmodule

module bnn_benchmark_baseline (
    input wire clk, rst_n, valid_in,
    input wire [39:0] input_features,
    output wire valid_out, fall_detected
);
    bnn_benchmark_shell #(.PIPELINED(0)) shell (
        .clk(clk), .rst_n(rst_n), .valid_in(valid_in),
        .input_features(input_features),
        .valid_out(valid_out), .fall_detected(fall_detected)
    );
endmodule

module bnn_benchmark_pipelined (
    input wire clk, rst_n, valid_in,
    input wire [39:0] input_features,
    output wire valid_out, fall_detected
);
    bnn_benchmark_shell #(.PIPELINED(1)) shell (
        .clk(clk), .rst_n(rst_n), .valid_in(valid_in),
        .input_features(input_features),
        .valid_out(valid_out), .fall_detected(fall_detected)
    );
endmodule
```